IdentityHashMap<K,V> [Comparing the keys based on memory reference OR address]

Interview Points :

* 1.4V
* Used to compre the **keys** based on **== operator** (Memory Address)
* does not use hashCode() OR equals(Object obj)
* For key comparison == and to find the flat array location It uses System.identityHashCode()
* Default capacity 32
* It uses flat array concept so It does not contain load factor specific constructor

public class IdentityHashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>, Clonable, Serializable.

It was introduced from JDK 1.4 onwards.

The IdentityHashMap uses == operator to compare key obejct.

HashMap uses hashCode() & equals() method.

We should use IdentityHashMap where we need to compare the keys by using reference or memory address instead of logical equality.

HashMap uses hashCode of the "Object key" to find out the bucket loaction in Hashtable, on the other hand IdentityHashMap does not use hashCode() method actually It uses System.identityHashCode(Object o)

IdentityHashMap is more faster than HashMap in case of key Comparison.

The default capacity is 32.

It has three constrcutors, It does not contain loadFactor specific constructor bacause It uses flat array concept.

1) IdentityHashMap ihm1 = new IdentityHashMap();
   Will create IdentityHashMap with default capacity is 32.

2) IdentityHashMap ihm2 = new IdentityHashMap(int initialCapacity);
   Will create IdentityHashMap with user specified capacity

3) IdentityHashMap ihm3 = new IdentityHashMap(Map map)

```
package com.ravi.identity_hash_map_demo;

import java.util.HashMap;
import java.util.IdentityHashMap;

public class IdentityHashMapDemo1
{
    public static void main(String[] args)
    {
       HashMap<String,Integer> hashMap = new HashMap<>();
       hashMap.put("Java", 1);
       hashMap.put(new String("Java"), 2);
       IO.println("Size of the HashMap is :"+hashMap.size()); //1
       IO.println(hashMap);
       IO.println("Identity Hash Map ....");
       IdentityHashMap<String,Integer> ihm = new IdentityHashMap<>();
       ihm.put("Java", 1);
       ihm.put(new String("Java"), 2);
       IO.println("Size of the IdentityHashMap is :"+ihm.size()); //2
       IO.println(ihm);

    }

}
```

Methods of SortedSet interface :

---

public E first() :- Will fetch first element

public E last() :- Will fetch last element

public SortedSet headSet(E element) :- Will fetch the values which are less than specified range.

public SortedSet tailSet(E element) :- Will fetch the values which are equal and greater than the specified range.

public SortedSet subSet(E startRange, E endRange) :- Will fetch the range of values where startRange is inclusive and endRange is exclusive.

Note :- headSet(), tailSet() and subSet(), return type is SortedSet.

```
import java.util.*;
public class SortedSetMethodDemo
{
      public static void main(String[] args)
      {
          TreeSet<Integer> times = new TreeSet<>();
          times.add(1205);
          times.add(1505);
          times.add(1545);
                times.add(1600);
          times.add(1830);
          times.add(2010);
          times.add(2100);


                IO.println("HeadSet Values are :"+times.headSet(1600));

                IO.println("TailSet Values are :"+times.tailSet(1600));

          SortedSet<Integer> sub = new TreeSet<>();

          sub = times.subSet(1545, 2010);

          IO.println("Sub set first value is :"+sub.first());
          IO.println("Sub set last value is :"+sub.last());

          IO.println("Subset Values are :"+sub);
           }
}
```

By using above SortedSet<E> methods we can find the range of values but navigation among the element is not possible.

In order to provide navigation facility among the elements, Java software people has introduced NavigableSet(I) interface from JDK 1.6V.

NavigableSet<E> :

It is a predefined interface available in java.util package from JDK 1.6V

It is sub interface of SorteSet<E>

It is used to navigate among the elements, Unlike SortedSet which provides range of values. Here we can navigate among the values as shown below.

```
 import java.util.*;

 public class NavigableSetDemo
 {
    public static void main(String[] args)
    {
       NavigableSet<Integer> ns = new TreeSet<>();
       ns.add(1);
       ns.add(2);
       ns.add(3);
       ns.add(4);
       ns.add(5);
       ns.add(6);

           IO.println("lower(3): " + ns.lower(3));//Just below than the specified element or null


      IO.println("floor(3): " + ns.floor(3)); //Equal  less or null


      IO.println("higher(3): " + ns.higher(3));//Just greater than specified element or null


     IO.println("ceiling(3): " + ns.ceiling(0));//Equal or greater or null

      }
 }
```

Methods of SortedMap<K,V> interface :

---

1) K firstKey() //first key (not the complete entry)

2) K lastKey() //last key

3) headMap(E keyRange) //less than the specified range (Entry)

4) tailMap(E keyRange) //equal or greater than the specified range

5) subMap(E startKeyRange, E endKeyRange) //the range of keys where startKey will be inclusive and endKey will be exclusive.

return type of headMap(), tailMap() and subMap() return type would be SortedMap(I)

```
import java.util.*;
public class SortedMapMethodDemo
      {
 public static void main(String args[])
      {
          SortedMap<Integer,String> map=new TreeMap<>();
            map.put(100,"Amit");
            map.put(101,"Ravi");

            map.put(102,"Vijay");
            map.put(103,"Rahul");

            System.out.println("First Key: "+map.firstKey());  //100
             System.out.println("Last Key "+map.lastKey());   //103

             System.out.println("headMap: "+map.headMap(102));  //100 101
             System.out.println("tailMap: "+map.tailMap(102));  //102 103
             System.out.println("subMap: "+map.subMap(100, 102)); //100 101

      }
 }
```

Assignment for NavigableMap Methods :

---

1) ceilingEntry(K key)
2) ceilingKey(K key)
3) floorEntry(K key)
4) floorKey(K key)
5) higherEntry(K key)
6) higherKey(K key)
7) lowerEntry(K key)
8) lowerKey(K key)

Properties :

---

public class Properties extends Hashtable<K,V>

It is a legacy class and It represents a persistent set of properties.

It is subclass of Hashtable available in java.util package.

It is used to maintain the persistent data in the key-value form. It takes both key and value as a String format.

**It is used to load properties file in our java application directly at RUNTIME without re-compilation/re-deployment/ and re-starting the server.

Constructors :

Commonly we are using this constructor :

   Properties p1 = new Properties();
   Creates an empty property list.

Methods :

---

1) public void load(InputStream stream): Reads a property list (key and value pair) from the input byte/binary stream.

2) public String getProperty(String key) : It is used to fetch the value based on the given key [If key is not matching then It will return null]

Steps :

---

1) Create a file whose extension must be .properties as shown below

database.properties

```
driverName = oracle.jdbc.OracleDriver
userName = scott
password = ravi
```

PropertyDemo1.java

```
import java.io.*;
import java.util.*;

public class PropertyDemo1
{
      public static void main(String[] args) throws Exception
      {
            var filePath = "D:\\new\\db.properties";
      var fin = new FileInputStream(filePath);

            var properties = new Properties();
            properties.load(fin);  //load(InputStream strm)

            String driver = properties.getProperty("driver");
            String userName = properties.getProperty("user");
            String pwd = properties.getProperty("password");

            IO.println("Driver Name is :"+driver);
            IO.println("UserName is :"+userName);
            IO.println("Password is :"+pwd);
      }
}
```

If we make changes in the database.properties file then directly (without compilation) we can take the the value in our java file so after any modification in the properties file we need not to re-compile/re-deploy our java program.

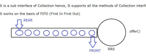
==

Queue<E>

It is a predfined sub interface of Collection available from JDK 1.5V

It is a sub interface of Collection hence, It supports all the methods of Collection interface

It works on the basis of FIFO (First In First Out)



It is an ordered collection.

In a queue, insertion is possible from last is called REAR end whereas deletion is possible from the starting is called FRONT end of the queue.

In order to support Basis Queue operation, LinkedList class implements Deque and Deque interface exetnds Queue interface.